Gym Battle – Produkt- und UX-Konzept

Stand: 28. Oktober 2026
Ziel: Umsetzungsgrundlage für einen Coding-Agenten. Dieses Dokument beschreibt Produktumfang, UX, Datenmodell und Sicherheitsanforderungen, enthält aber keinen Anwendungscode und keine Deployment-Schritte.

0. Produktziel und explizite Annahmen

Produktziel: Freunde sollen ein Krafttraining schnell protokollieren und sich in privaten Gruppen zu regelmäßigen Trainings motivieren können. Der soziale Teil ist bewusst klein, opt-in und gruppenbasiert. Das Log muss auch ohne Gruppe brauchbar sein.

Annahmen – zur Bestätigung durch das Produktteam, keine vorgegebenen Fakten:

1. Erste Version ist eine responsive Web-App, zunächst online nutzbar. PWA-Installation und Offline-Logging sind nicht Teil der ersten Lieferung.
2. UI-Sprache ist zunächst Deutsch. Datum, Wochenstart und Zahlenformat folgen dem deutschen Locale; Gewichte werden intern in kg gespeichert, die Anzeige kann später lb unterstützen.
3. Anmeldung erfolgt mit E-Mail und Passwort über Supabase Auth. E-Mail-Bestätigung und Passwort-Reset gehören zum Auth-Flow. Ein separater kostenpflichtiger E-Mail-Dienst wird für Einladungen nicht vorausgesetzt.
4. Freundschaft und Gruppenmitgliedschaft sind getrennte Konzepte: Eine Freundschaft ist gegenseitig bestätigt; ein Gruppenlink kann direkt eine Gruppeneinladung sein. Nutzer können außerdem Freunde aus ihrer Freundesliste in eine Gruppe einladen.
5. Ein abgeschlossenes Training ist standardmäßig privat. Um in einer Gruppenrangliste zu zählen, wählt der Nutzer beim Abschluss genau eine Gruppe, mit der dieses Training geteilt wird. Ein Training in mehreren Gruppen gleichzeitig ist nicht Teil des MVP.
6. Die Rangliste belohnt Trainingshäufigkeit, nicht Gewicht, Volumen oder Körperdaten. Ein abgeschlossener Trainingstag zählt als ein Punkt; pro Nutzer und Gruppenkalenderwoche zählen maximal fünf Trainingstage.
7. Gruppen sind privat und nur für Mitglieder sichtbar. Es gibt keine globale Suche, keinen öffentlichen Feed, keine Direktnachrichten und keine öffentlichen Profile.
8. Das MVP verwendet einen kleinen eingebauten Übungskatalog plus eigene, private Übungen. Eine umfangreiche, medizinisch geprüfte Bibliothek wird nicht vorausgesetzt.

1. Informationsarchitektur und Navigation

Sitemap

Plain Text

```
/
├── /auth/sign-in
├── /auth/sign-up
├── /auth/forgot-password
├── /auth/reset-password
├── /auth/callback
├── /onboarding/profile
└── /app
    ├── /dashboard
    ├── /workouts
    │   ├── /new
    │   └── /[workoutId]
    ├── /history
    │   └── /[workoutId]
    ├── /groups
    │   ├── /new
    │   ├── /[groupId]
    │   │   ├── /leaderboard
    │   │   ├── /members
    │   │   └── /settings
    │   └── /invite/[token]
    ├── /friends
    ├── /profile
    └── /settings
```

Primäre Navigation

Mobile: feste Bottom-Navigation mit vier Zielen: **Übersicht**, **Training**, **Gruppen**, **Profil**. „Training starten“ erhält zusätzlich einen auffälligen, gut erreichbaren CTA auf der Übersicht und der Trainingshistorie. Freundschaften befinden sich auf der Gruppen-Seite in einem sekundären Tab bzw. einer Unterseite, nicht als fünfter Bottom-Tab.

Desktop: schmale linke Seitenleiste mit denselben vier Zielen; Gruppen-Unterseiten zeigen eine zweite, lokale Navigation mit „Übersicht“, „Rangliste“, „Mitglieder“ und – für Admins – „Einstellungen“. Hauptinhalt bleibt auf eine gut lesbare Maximalbreite begrenzt.

Navigationsregeln:

- Nicht angemeldete Nutzer werden von `/app/*` zu `/auth/sign-in` geleitet. Nach erfolgreicher Anmeldung geht es zurück zum ursprünglich gewünschten Ziel.
- Neue Nutzer durchlaufen einmalig Profil-Onboarding; ein valider Benutzername und Anzeigename sind Voraussetzung für Gruppen und Ranglisten.

- Ein gültiger Gruppen-Einladungslink darf vor der Anmeldung eine Vorschau zeigen. Die Mitgliedschaft wird erst nach Anmeldung und aktiver Bestätigung erstellt.
- Browser-Zurück soll im Training nicht versehentlich ein laufendes Training verwerfen. Beim Verlassen mit ungespeicherten Änderungen erscheint ein Bestätigungsdialog.

2. Nutzerabläufe

A. Registrierung und erstes Öffnen

1. Landing/Anmeldeansicht bietet „Konto erstellen“ und „Anmelden“; Gruppenlink kann als Return-URL erhalten bleiben.
2. Nutzer gibt E-Mail, Passwort und Passwortbestätigung ein und akzeptiert Nutzungsbedingungen/Datenschutzhinweis, sofern diese in der Produktrealisierung bereitgestellt werden.
3. Supabase Auth erstellt die Identität. Bei aktivierter Bestätigung wird eine klare „Prüfe deine E-Mail“-Ansicht gezeigt; erneutes Senden ist möglich und gedrosselt.
4. Nach Authentifizierung legt der Nutzer Anzeigename und eindeutigen Handle fest. Avatar ist optional; Standard ist ein Initialen-Avatar.
5. Optionales Zielmenü: „Gruppe erstellen“, „Einladung öffnen“ oder „Erst einmal allein trainieren“. Keine erzwungene Kontaktbuch-Synchronisierung.
6. Weiter zur Übersicht mit einer kurzen Erklärung der privaten Gruppen und des privaten Trainingslogs.

Fehler- und Randfälle: E-Mail schon vorhanden, ungültiges Passwort, Bestätigungslink abgelaufen, Handle vergeben, Netzwerkfehler, Nutzer öffnet Invite-Link auf anderem Gerät. In jedem Fall bleibt eine klare nächste Aktion sichtbar.

B. Private Gruppe erstellen und Freunde einladen

1. Auf „Gruppen“ tippt der Nutzer auf **Gruppe erstellen**.
2. Formular: Gruppenname (erforderlich), optional Beschreibung, Zeitzone (Standard aus Profil/Browsereinstellung, editierbar). Sichtbarkeit ist fest „Privat“.
3. Nach Erstellung wird der Ersteller Mitglied mit Rolle `owner` und landet auf der Gruppenübersicht.
4. „Freunde einladen“ bietet zwei Wege: vorhandenen bestätigten Freund auswählen oder Einladungslink erstellen/kopieren/teilen. Keine Einladung wird ohne ausdrückliche Aktion verschickt.

5. Empfänger öffnet Vorschau mit Gruppenname, Ersteller und ungefährem Mitgliederstand; private Inhalte bleiben verborgen. Nach Anmeldung/Bestätigung wird die Mitgliedschaft aktiviert.
6. Die Gruppe erscheint für beide Nutzer in der Gruppenliste. Der Owner sieht einen Hinweis, falls noch keine Mitglieder oder Trainings vorhanden sind.

Einladungsregeln: Einladungslink kann widerrufen werden; im MVP läuft er nach 7 Tagen ab und ist für eine Person nutzbar. Wiederverwendbare Gruppenlinks können später ergänzt werden. Diese Default-Werte sind eine Produktannahme und in einer zentralen Einstellung konfigurierbar.

C. Training protokollieren

1. Nutzer wählt **Training starten**.
2. Optional: bestehende Routine wählen oder **Leeres Training** starten. MVP darf mit leerem Training funktionieren; eine ausgefeilte Routineverwaltung ist nicht Voraussetzung.
3. Aktive Trainingsansicht: Trainingsdauer, Liste der Übungen, **Übung hinzufügen**, pro Übung schnell editierbare Sätze. Ein Satz erfasst Gewicht (kg) und Wiederholungen; Satz hinzufügen, duplizieren und entfernen.
4. Vorheriger Leistungswert der Übung wird neben der Eingabe gezeigt, falls vorhanden. Standardwerte können aus der letzten abgeschlossenen eigenen Session vorbelegt, aber nicht stillschweigend als erledigt gespeichert werden.
5. Änderungen werden nach Eingabe bzw. Satzbestätigung automatisch gespeichert. Sichtbarer Status: „Wird gespeichert “, „Gespeichert “ oder „Nicht gespeichert – erneut versuchen “.
6. Nutzer tippt **Training beenden**. Bestätigungsansicht zeigt Übungs-/Satzanzahl, Dauer, optionalen Titel und Teilungswahl: **Privat** oder **mit [Gruppe] teilen**.
7. Bei Gruppenfreigabe erklärt die UI genau, welche Mitglieder Trainingsname, Übungen, Sätze, Gewicht, Wiederholungen und Abschlusszeit sehen. Nutzer bestätigt.
8. Training wird abgeschlossen, ein Erfolgszustand zeigt Zusammenfassung, neue persönliche Bestleistungen (nur wenn zuverlässig errechenbar) und Gruppenergebnis.

Abbruch/Recovery: Laufendes Training bleibt als Entwurf erhalten und erscheint als „Training fortsetzen “. Abbrechen löscht erst nach Bestätigung. Bei Netzwerkfehlern werden Eingaben nicht still verworfen; Speichern und Abschluss bleiben gesperrt bzw. in retry-fähigem Zustand.

D. Gruppenrangliste ansehen

1. Nutzer öffnet eine Gruppe und wählt **Rangliste**.
2. Standardzeitraum ist die laufende Gruppenkalenderwoche. UI zeigt Datumsbereich, Zeitzone und Punkte-Regel: ein freigegebener Trainingstag = 1 Punkt, maximal 5 Punkte pro Woche.
3. Mitglieder werden nach Punkten absteigend angezeigt. Gleichstand erhält denselben Rang; sekundär wird alphabetisch nach Anzeigename sortiert, nicht nach Trainingslast.
4. Zeile zeigt Rang, Avatar/Initialen, Anzeigename, Punkte und Anzahl unterschiedlicher Trainingstage. Details eines Trainings sind nur sichtbar, wenn es für diese Gruppe geteilt wurde.
5. Nutzer kann vorherige Woche ansehen. Im MVP werden frühere Wochen nur aus den zugänglichen abgeschlossenen, geteilten Trainings berechnet; keine manuelle Ergebnisänderung.
6. Empty state erläutert die Regel und bietet „Training protokollieren “. Ranglistenplatzierung enthält keine beschämenden Warnungen oder negativen/roten „Verlierer “-Botschaften.

3. Seiten, Inhalte und UI-Zustände

Seite	Kerninhalt und Aktionen	Wesentliche UI-Zustände
Sign-in	E-Mail, Passwort, Anmelden, Passwort vergessen, Konto erstellen.	Laden; Validierungsfehler am Feld; falsche Zugangsdaten mit neutraler Meldung; Netzwerkfehler mit Retry; Erfolg/Weiterleitung.
Sign-up	E-Mail, Passwort, Bestätigung, Nutzungsbedingungen-Verweis.	Passwortregeln; E-Mail bereits registriert; Bestätigung ausstehend; Bestätigung erneut senden; Rate-limit-Hinweis.
Forgot/Reset password	E-Mail zum Zurücksetzen bzw. neues Passwort aus Link.	Unbekannte/ungültige Anfrage ohne unnötige Kontooffenlegung; Link abgelaufen; Passwort erfolgreich geändert.
Profil-Onboarding	Anzeigename, eindeutiger Handle, optional Avatar.	Handle verfügbar/vergeben/ungültig; Upload-Fehler; überspringbare Avatar-Auswahl.

Übersicht	Willkommen, CTA „Training starten “, laufendes Training fortsetzen, nächste Challenge/Wochenfortschritt, zuletzt absolvierte Sessions, Gruppen-Kurzliste.	Erstnutzer ohne Gruppe; kein Training bisher; laufender Entwurf; Gruppenladung; Datenfehler und Retry. Keine gefälschten Demo-Werte.
Training starten/Editor	Routine oder leer; Übungssuche; Übungsliste; Satzeditor; Timer optional.	Leeres Training; Suche ohne Treffer; Übung hinzufügen; Satz leer/ungültig; Speichern; offline/Fehler; Entwurf wiederherstellen; Abbruchdialog.
Training abschließen	Zusammenfassung und Freigabewahl.	Keine Übungen/Sätze; Freigabe-Dialog; Abschluss läuft; Fehler beim Speichern; Erfolg.
Training-Detail	Datum, Dauer, Übungen, Sätze, ggf. persönliche PR-Markierung und geteilte Gruppe.	Privatansicht des Besitzers; Gruppenansicht mit nur freigegebenen Daten; nicht autorisiert/gelöscht.
Historie	Chronologische Liste, Datumsfilter einfach, Trainingszusammenfassung, Suchfeld optional.	Erster Eintrag fehlt; keine Treffer; paginiert/lädt mehr; Fehler. MVP braucht keine komplexe Analyse- oder Kalendervisualisierung.
Gruppenliste	Eigene Gruppen, Mitgliederanzahl, aktuelle Woche, Gruppe erstellen, Freunde verwalten.	Noch keine Gruppe; Ladezustand; Gruppe archiviert; Fehler.
Gruppe erstellen	Name, optional Beschreibung, Zeitzone, feste private Sichtbarkeit.	Feldvalidierung; Handle nicht relevant; Speichern; doppelte/fehlgeschlagene Anfrage.
Gruppenübersicht	Name, aktuelle Gruppenwoche, Ranglisten-Vorschau, Mitglieder, Einladen, Trainings teilen.	Neue leere Gruppe; nur Owner; Mitgliedschaft entzogen; Gruppe archiviert; fehlende Berechtigung.
Rangliste	Zeitraumumschalter Woche aktuell/vorherig, Regelhinweis, geordnete Mitgliederliste.	Noch keine geteilten Trainings; Gleichstände; Nutzer außerhalb der Gruppe; Abfragefehler; Daten werden geladen.

Mitglieder	Mitgliederliste, Rollen, Einladung; Owner kann Admins verwalten und Gruppe archivieren.	Einladung erstellt/widerrufen; Nutzer ausgetreten; letzte Owner-Änderung; Berechtigungsfehler.
Invite-Vorschau	Gruppenname, Einladender, Ablaufhinweis, Beitreten/Anmelden. Keine privaten Posts/Trainings.	Token gültig/abgelaufen/widerrufen/verbraucht; bereits Mitglied; Anmeldung nötig; Gruppe voll, falls später eingeführt.
Freunde	Suche nach exaktem Handle/Prefix, eingehende Anfragen, ausgehende Anfragen, bestätigte Freunde.	Keine Freunde; keine Treffer; Anfrage ausstehend; akzeptiert/abgelehnt; Ratelimit; keine E-Mail-Enumeration.
Profil	Anzeigename, Handle, eigene Trainingszusammenfassung; Profilbearbeitung. Standardmäßig nur für Freunde und gemeinsame Gruppen sichtbar.	Eigenes Profil; fremdes Profil mit eingeschränkten Feldern; nicht sichtbar; fehlender Avatar.
Einstellungen	Locale/Einheiten, Zeitzone, Privatsphäre, Benachrichtigungen, Passwort/Abmelden, Konto löschen.	Speichern; Bestätigung für irreversible Kontoaktion; Fehlermeldungen; abgelaufene Sitzung.
404/Fehler	Knapp erklären, dass Seite nicht verfügbar ist; zur Übersicht oder Gruppenliste zurück.	Nicht zwischen „existiert nicht“ und „keine Berechtigung“ unterscheiden, wenn das Seiten-Dasein sonst private Informationen verraten würde.

Gemeinsame UX-Zustände

- Skeletons statt Springen im Layout; Ladezustand für Listen und Aktionen getrennt.
- Leere Zustände erklären den nächsten sinnvollen Schritt und zeigen eine primäre CTA.
- Fehlertexte nennen Ursache nur, wenn sie sicher bekannt ist; Retry ist bei transienten Fehlern sichtbar.
- Toasts nur für kurze Bestätigung; wichtige Speicherung zusätzlich dauerhaft am Inhalt anzeigen.

- Destruktive Aktionen wie Training löschen, Gruppe archivieren oder Freund entfernen brauchen Bestätigung und nennen die Auswirkung.
- Formulare behalten Eingaben bei Fehlern. Alle Aktionen sind gegen Doppelklick/mehrfaches Absenden abgesichert.
- Tastaturbedienbarkeit, sichtbarer Fokus, Screenreader-Labels und ausreichender Farbkontrast von Anfang an einplanen.

4. Designsystem

Visuelle Richtung

Annahme: hochwertige, sportliche Oberfläche ohne überladene Gaming-Optik. Dunkle Anthrazitflächen bilden die ruhige Basis; ein leuchtendes Lime setzt gezielte Akzente für Fortschritt und primäre Aktionen. Kein permanentes Neon, keine aggressiven „Gewinner/Verlierer“-Farben.

Farbpalette

Token	Wert	Verwendung
`background`	`#0B0F0D`	Seitenhintergrund in Dark Theme
`surface`	`#141A17`	Karten, Navigation
`surface-raised`	`#1C2420`	Dialoge, Hover, Eingabeflächen
`border`	`#2A342E`	Trennlinien, Umrandungen
`text-primary`	`#F4F7F4`	Haupttext
`text-secondary`	`#A6B2AA`	Hinweise, Labels
`accent`	`#B7F34A`	Primärbutton, aktive Navigation, Fortschritt
`accent-pressed`	`#9BD52E`	gedrückter Zustand
`success`	`#44D19A`	gespeichert/abgeschlossen
`warning`	`#F4C66A`	Ablauf/aufmerksam machen
`danger`	`#FF7474`	destruktive Aktion/Fehler

Farbe nie als einziges Signal verwenden: Icon + Text/Statuslabel hinzufügen. Lime für Text nur bei ausreichendem Kontrast nutzen; im Zweifel dunkle Schrift auf Lime-Fläche.

Typografie und Layout

- Schrift: system-ui oder eine lokal gebündelte variable Sans-Schrift; keine externen kostenpflichtigen Font-CDNs. Zahlen in Trainingslogs mit tabellarischen Ziffern.
- Typografieskala: 12 px Caption, 14 px Label, 16 px Body, 20 px H3, 24 px H2, 32 px H1. Body-Zeilenhöhe 1.45–1.6.
- 4-px Basiseinheit; häufige Abstände 8/12/16/24/32/48 px.
- Mobile Inhaltspadding 16 px, Desktop 24 px; Maximalbreite für Hauptinhalte ca. 1120 px.
- Radius 8 px für Inputs, 12 px für Karten, 16 px für große Panels; Buttons mindestens 44 px hoch, Touch-Ziele mindestens 44×44 px.
- Responsive Breakpoints orientieren sich an Tailwind-Defaults; mobile zuerst. Satzeditor auf schmalen Displays stapeln (Satznummer, Gewicht, Wiederholungen), nicht zu kleine Tabellenfelder.

Komponentenbibliothek (MVP)

`Button` (primary/secondary/quiet/danger), `IconButton`, `TextField`, `PasswordField`, `NumberField`, `Select`, `Checkbox`, `Avatar`, `Badge`, `Card`, `Tabs`, `BottomNavigation`, `Sidebar`, `Dialog/Drawer`, `Toast`, `Skeleton`, `EmptyState`, `ErrorState`, `WorkoutSetRow`, `ExerciseCard`, `LeaderboardRow`, `InviteLinkPanel`, `SaveStatus`.

Komponentenregeln: Fokuszustand und disabled/loading für alle interaktiven Komponenten; Zahleneingaben erlauben Dezimal-Komma in deutscher UI und normalisieren intern; Gewicht kann 0 für Körpergewichtsübungen sein, Wiederholungen sind positive ganze Zahlen. Kein Button darf im Ladezustand mehrfach auslösen.

5. PostgreSQL-Datenmodell

Alle Tabellen liegen in einem nicht direkt öffentlich erreichbaren Anwendungsschema (z. B. `public` mit konsequent aktivierter RLS). Primärschlüssel UUID. Zeitpunkte als `timestamptz` in UTC; Gruppenkalender verwendet IANA-Zeitzone aus `groups.timezone`. Referenzielle Löschregeln explizit setzen; kein stilles Löschen historischer Trainings durch Gruppenlöschung.

Entitäten

Tabelle	Wichtige Felder	Beziehungen / Regeln

profiles	id (PK/FK auth.users.id), username (unique, normalisiert/lowercase), display_name, avatar_path nullable, timezone, weight_unit, created_at, updated_at	1:1 zu Auth User. E-Mail bleibt in auth.users, nie in Profil-/Gruppensuche offenlegen.
friendships	id, user_low_id, user_high_id, initiated_by, status (pending, accepted), created_at, updated_at	Beide User-IDs kanonisch sortiert; unique Paar; Initiator muss Teilnehmer sein; keine Selbstfreundschaft. decline / remove löscht oder beendet die Beziehung durch kontrollierte Aktion.
groups	id, name, description nullable, created_by, timezone, created_at, archived_at nullable	Creator wird beim Erstellen als Owner-Mitglied eingetragen. Gruppe privat, keine Public-Flag im MVP.
group_members	group_id, user_id, role (owner, admin, member), status (active, left, removed), joined_at	PK/unique (group_id,user_id); aktive Mitglieder sehen Gruppeninhalte. Genau ein Owner, Übertragung nur über kontrollierte Aktion.
group_invites	id, group_id, created_by, token_hash, target_user_id nullable, expires_at, max_uses, use_count, revoked_at, created_at	Token wird nur gehasht gespeichert; niemals Hash/Token an beliebige Gruppenmitglieder zurückgeben. Gültigkeit und Nutzung atomar prüfen.
exercises	id, owner_id nullable, name, muscle_group nullable, equipment nullable, is_system, created_at	Systemübungen sind für angemeldete Nutzer lesbar. Eigene Übungen sind im MVP privat und nur ihrem Ersteller sichtbar. Unique (normalisierter Name, Owner) sinnvoll.

workout_sessions	id , user_id , title nullable, status (in_progress , completed , cancelled), started_at , completed_at nullable, shared_group_id nullable, created_at , updated_at	Besitzer ist Auth-User. shared_group_id nur bei aktiver Mitgliedschaft des Besitzers; sonst NULL . Eine Session teilt sich im MVP höchstens mit einer Gruppe.
workout_exercises	id , workout_session_id , exercise_id nullable, exercise_name_snapshot , position , notes nullable	Geordnete Übungen innerhalb einer Session; Snapshot erhält Historie bei Katalogänderung. Parent-Session muss dem Nutzer gehören.
workout_sets	id , workout_exercise_id , position , weight_kg numeric nullable, reps integer nullable, is_completed , created_at	Satz gehört genau zu einer Übung. Mindestens Gewicht oder Wiederholungen je Satz zulässig; für MVP Logging-Regel Gewicht>=0 und Reps>0, wenn Satz abgeschlossen. Reihenfolge eindeutig pro Übung.

Beziehungen (Kurzform)

- auth.users 1—1 profiles
- profiles N—N profiles über friendships
- profiles N—N groups über group_members
- groups 1—N group_invites
- profiles 1—N workout_sessions
- groups 1—N workout_sessions über optionale shared_group_id
- workout_sessions 1—N workout_exercises 1—N workout_sets
- exercises 1—N workout_exercises ; gelöschte eigene Übungen hinterlassen den Snapshot-Namen.

Ranglistenberechnung

Keine vorab gespeicherten Punkte im MVP. Für eine Woche in groups.timezone werden abgeschlossene Sessions mit shared_group_id = group_id gezählt. Ein Mitglied erhält maximal einen Punkt pro lokalem Gruppenkalendertag und maximal fünf Punkte pro Woche. completed_at definiert den Zähltag. Gleichstände teilen den Rang. workout_sessions -Details

erscheinen nur, wenn die Mitgliedschaft aktiv ist und die Session ausdrücklich für diese Gruppe freigegeben wurde.

Rangliste zunächst serverseitig aus einer aggregierten SQL-Abfrage bzw. einer RLS-sicheren Datenbankfunktion berechnen; erst bei nachgewiesenem Bedarf materialisieren. Keine clientseitige Summierung aus Daten, auf die der Nutzer keinen Lesenzugriff haben darf.

Indizes und Integrität

- Indexe auf `group_members(user_id,status)`, `group_members(group_id,status)`, `workout_sessions(user_id,completed_at)`, `workout_sessions(shared_group_id,completed_at)`, `workout_exercises(workout_session_id,position)`, `workout_sets(workout_exercise_id,position)`.
- Check constraints für gültige Status-/Rollenwerte, `weight_kg >= 0`, `reps > 0` bei abgeschlossenen Sätzen, `expires_at > created_at`, `max_uses > 0`.
- Unique Constraints gegen doppelte Freundschaftspaare, doppelte Gruppenmitgliedschaften und doppelte Übungs-/Satzpositionen.
- Erstellen von Gruppe + Owner-Mitgliedschaft sowie Akzeptieren eines Gruppeninvites + Zählererhöhung müssen atomar sein.

6. Sicherheit und Supabase-RLS

Sicherheitsgrundsätze

1. RLS für jede Tabelle aktivieren, die über Supabase-APIs erreichbar ist. Standard ist „deny “, danach gezielt erlauben.
2. `service_role` ausschließlich in serverseitiger Laufzeit/Secret Store, niemals in Browser-Bundle, Client-Komponente oder URL.
3. Authentifizierte Identität aus der validierten Supabase-Session beziehen; niemals `user_id` aus einem Formular als Berechtigungsbeweis verwenden.
4. RLS ist die Sicherheitsgrenze, UI-Filter sind nur UX. Server Actions/Route Handlers validieren zusätzlich Eingaben und Zustandsübergänge.
5. Gruppenbeitritt und Einladungstoken serverseitig prüfen. Token stark zufällig, einmalig/nutzungsbegrenzt, gehasht gespeichert, widerrufbar und befristet. Akzeptanz rate-limiten.
6. Keine E-Mail-Suche, keine Kontoexistenzbestätigung bei Passwort-Reset, keine öffentliche Auflistung aller Profile. Nutzer finden Freunde nur über exakten Handle/kleine begrenzte Suche.
7. Nutzer können nur eigene Sessions und untergeordnete Satzdaten bearbeiten. Nach Abschluss sind Sessiondaten im MVP nur vom Eigentümer änderbar; Änderungen an

geteilten Daten müssen als Eigentümeraktion erfolgen.

8. Datenschutzfreundliche Logs: keine Passwörter, Session-Tokens, Einladungs-URLs oder Trainingsdetails in Fehler-/Analytics-Logs.
9. Konto löschen muss Sessions, Freundschaften und Mitgliedschaften nach dokumentierter Löschregel behandeln; geteilte Gruppenhistorie darf nicht versehentlich personenbezogene Daten weiter zeigen. MVP kann Anonymisierung des Profilnamens für historische Gruppenpunkte definieren.

RLS-Policy-Matrix

Tabelle	Lesen	Einfügen	Ändern/Löschen
`profiles`	Nutzer sieht eigenes Profil; eingeschränkte öffentliche Felder nur bei bestätigter Freundschaft oder gemeinsamer aktiver Gruppe. Handle-Suche über begrenzte Funktion, nie rohe Volltabellenabfrage.	Eigenes Profil nur mit `id = auth.uid()`; Profilanlage nach Registrierung serverseitig/Trigger.	Nur eigenes Profil und nur erlaubte Profilfelder. Rolle/ID nicht änderbar.
`friendships`	Nur Teilnehmer der Beziehung.	`initiated_by = auth.uid()`, eigener User ist Teilnehmer, kein Duplikat/Selbstlink.	Nur Empfänger kann `pending` → `accepted` setzen; Delete nur durch Beteiligten. Initiator und User-IDs danach unveränderlich.
`groups`	Aktive Mitglieder; nicht Mitglieder erhalten keine Gruppenmetadaten außer einer minimalen geprüften Invite-Vorschau.	Authentifizierter Ersteller; Owner-Mitgliedschaft in derselben atomaren Transaktion anlegen.	Owner/Admin darf Name/Beschreibung/Zeitzone nach Policy ändern; Archivieren nur Owner.
`group_members`	Aktive Gruppenmitglieder sehen Mitglieder derselben Gruppe.	Ausschließlich geprüfter Invite-Flow; kein freies Self-Insert.	Nutzer darf eigenen Status auf `left` setzen; Owner/Admin darf regulär verwalten. Selbstbeförderung zu Admin/Owner verbieten. Owner-

			Wechsel nur atomar und kontrolliert.
`group_invites`	Owner/Admin verwalten Metadaten; Mitglieder dürfen höchstens einen Erstellungsdialog auslösen. Roher Token/Hash nie lesbar. Invite-Vorschau nur über validierende Funktion/Route.	Owner/Admin darf Invite mit `created_by = auth.uid()` erstellen.	Owner/Admin kann widerrufen; Nutzungszähler nur in atomarer Serverfunktion aktualisieren.
`exercises`	Systemkatalog für angemeldete Nutzer; eigene Übung nur Ersteller.	Eigene Übung nur mit `owner_id = auth.uid()` und `is_system = false`.	Nur Eigentümer; Systemdatensätze nur über privilegierten Wartungspfad.
`workout_sessions`	Eigentümer vollständig; Gruppenmitglieder nur `completed`-Session mit `shared_group_id` ihrer aktiven Gruppe. Private Session ausschließlich Eigentümer.	Nur für `user_id = auth.uid()`; `shared_group_id` muss aktive Mitgliedschaft aufweisen.	Nur Eigentümer; Wechsel von `user_id` verboten. Teilen/Entteilen nur als Eigentümer und aktive Mitgliedschaft. Löschen bevorzugt Soft-delete oder explizite Bestätigung.
`workout_exercises`	Nur wenn Parent-Session lesbar ist.	Nur wenn Parent-Session dem Auth-User gehört und bearbeitbar ist.	Gleiche Parent-Prüfung; keine Änderung an fremder Session.
`workout_sets`	Nur wenn Parent-Kette bis Session lesbar ist.	Nur wenn Parent-Session dem Auth-User gehört und `in_progress` ist.	Nur Besitzer der laufenden Session; abgeschlossene Sessions schreibgeschützt, außer explizite Owner-Korrektur.

Policy-Implementierungshinweise:

- RLS-Prüfungen für Gruppenmitgliedschaft können in eng begrenzten Datenbankfunktionen gekapselt werden, um Policy-Rekursion zu vermeiden. Falls

`SECURITY DEFINER` verwendet wird: festen `search_path` setzen, Ausführungsrechte einschränken, Auth-Identität selbst prüfen und nur minimale Funktion freigeben.

- Einladungsbeitritt soll eine einzige transaktionale Funktion/Route verwenden, die Token validiert, Status/Ablauf prüft, Membership anlegt und Zähler erhöht. Keine clientseitige Mehrschritt-Logik, die teilweise erfolgreiche Zustände hinterlässt.
- Tabellen dürfen keine Policy enthalten, die nur auf vom Client änderbare `role` - oder `user_id` -Werte vertraut. Für `UPDATE` sowohl `USING` (alter Zustand) als auch `WITH CHECK` (neuer Zustand) festlegen.
- Tests mit mindestens zwei Auth-Usern und getrennten Gruppen: jede Policy für erlaubten und verbotenen Zugriff testen, einschließlich direkter REST-Abfragen statt ausschließlich UI-Klickpfaden.

7. Implementierungsplan nach Abhängigkeiten

Phase 1 – Produktschnitt und Grundlagen

- Festschreiben der Annahmen: Ranglistenregel, Gruppenzeitzone, Einladungsablauf, Profile-Sichtbarkeit, ob Sessiondaten nach Abschluss editierbar sind.
- Supabase Projekt lokal/Entwicklungsumgebung, Schema-Migrationen und Testdaten einrichten. Migrationen versioniert halten.
- Datenmodell, Constraints, Indizes, RLS und minimal nötige Transaktionsfunktionen zuerst anlegen.
- Supabase Auth für E-Mail/Passwort plus Redirect-/Callback-Flow und Sessionhandling in Next.js SSR festlegen.

Abnahmekriterium: Tabellen und RLS-Tests belegen Isolation zwischen zwei Nutzern; kein Gruppen- oder Sessionzugriff ohne Mitgliedschaft/Eigentum.

Phase 2 – App-Shell und Auth

- App Router Layouts für Auth, Onboarding und geschützten App-Bereich.
- Responsive Navigation, Design-Tokens, Formular-/Button-/Dialog-/Toast-/Empty-State-Komponenten.
- Registrierung, Anmeldung, Abmelden, Passwort-Reset, Callback, Profil-Onboarding.
- Zugriffsprüfung serverseitig in geschützten Seiten; UI-Redirect allein ist kein Schutz.

Abnahmekriterium: Registrierter Nutzer kann Profil vervollständigen; nicht authentifizierter Request kann keine App-Daten lesen.

Phase 3 – Gruppen, Freundschaften und Einladungen

- Gruppen erstellen, listen, umbenennen, archivieren.
- Mitgliederliste/Rollen und Freundschaftsanfragen über Handle.
- Invite-Link erzeugen, Vorschau, Annahme, Ablauf und Widerruf.
- Einladungsbeitritt atomar und gegen doppelte/konkurrierende Requests abgesichert.

Abnahmekriterium: Private Gruppe ist für Nichtmitglieder auch per direkter API nicht sichtbar; nur gültiger Invite fügt Mitglied hinzu.

Phase 4 – Trainingslog

- System-Übungskatalog mit begrenzter Auswahl und eigener privater Übung.
- Session starten, Übungen hinzufügen, Sätze mit Gewicht/Wiederholungen editieren, Session autospeichern und fortsetzen.
- Session abschließen, privat lassen oder für genau eine Gruppe freigeben.
- Historienliste und Detailansicht.
- Fehler- und Doppelklickverhalten, Save-Status und fehlgeschlagene Speicherung testen.

Abnahmekriterium: Nutzer kann eine Session beginnen, schließen, später in History öffnen; fremde Nutzer können private Daten auch per Direktabfrage nicht sehen.

Phase 5 – Rangliste und Motivation

- Gruppenkalenderwoche aus `groups.timezone` berechnen.
- Rangliste serverseitig aggregieren; nur freigegebene Sessions zählen.
- Regeln sichtbar machen, Gleichstand behandeln und Vorwoche anzeigen.
- Übersichtskarte und leere Zustände anbinden.

Abnahmekriterium: Tests decken Wochenwechsel/Zeitzone, maximal einen Punkt pro Tag, Wochenlimit 5, Gleichstand, Session-Entteilen und Gruppenwechsel ab.

Phase 6 – Qualität und MVP-Härtung

- End-to-end-Flows: Registrierung, Invite-Annahme, Gruppe, Training, Freigabe, Rangliste, Verlassen der Gruppe.
- Responsive Prüfung bei schmalem Mobilviewport, Tablet und Desktop; Tastatur, Fokus, Kontrast und Screenreader-Basics prüfen.
- Fehlerfälle: abgelaufene Einladung, gelöschte Gruppe, veraltete Sitzung, Netzwerkabbruch während Speichern, doppelter Abschluss, gelöschte Übung.

- Sicherheitsprüfung aller RLS-Policies und Mutationen, Rate-Limits für Suche/Invite, keine Secrets im Client.
- Erst danach PWA-Manifest/Installierbarkeit; Offline-Logging, Push und Wearables bewusst aus dem ersten Release heraushalten.

8. MVP-Grenze und Nicht-Ziele

MVP enthält: Registrierung/Anmeldung, Profil/Handle, Freundschaftseinladungen, private Gruppen, Gruppenlink, Übungsauswahl, Gewicht/Wiederholungen pro Satz, Trainingshistorie, explizite Gruppenteilung und Wochenrangliste für Trainingskonsistenz.

MVP enthält nicht: globale Ranglisten, öffentlicher Social Feed, Chat, GPS, Gesundheitsdaten/Wearables, KI-Coach, Trainingsprogramme mit Periodisierung, Körpermaße, Social Login, komplexe Badges, Push-Nachrichten, Abo/Bezahlschranke, PWA-Offlinesync.

Produktprinzip: Ein Nutzer darf ohne Freunde und ohne Gruppe ein gutes Trainingslog erhalten. Der soziale Wettbewerb wird nur innerhalb der selbst gewählten Gruppe sichtbar und nur mit bewusst freigegebenen Trainingsdaten gespeist.